

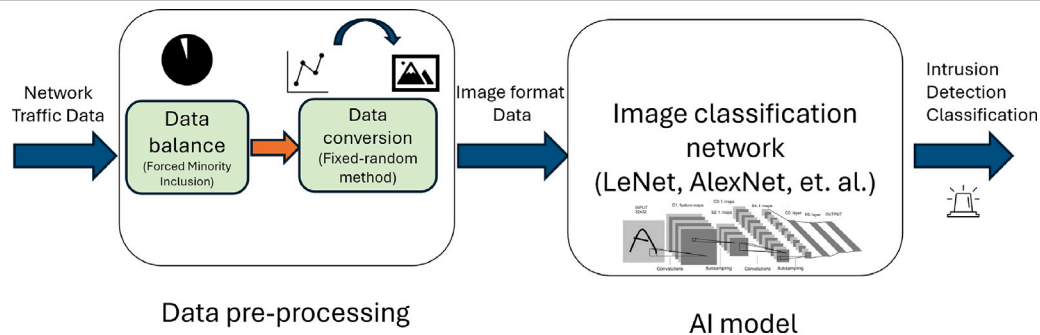
Intrusion detection in IoT and wireless networks using image-based neural network classification

Yanxia Sun^a, Zenghui Wang^b *

^a Department of Electrical and Electronic Engineering Science, University of Johannesburg, Johannesburg, 2006, South Africa

^b Department of Electrical and Smart Systems Engineering, University of South Africa, FL, 1709, South Africa

GRAPHICAL ABSTRACT



ARTICLE INFO

Keywords:

Intrusion detection
Image processing
LeNet
Machine learning
Wireless network
Internet of Things
Deep learning

ABSTRACT

Telecommunication networks play more and more important role in our modern times, and there are significant security risks associated with both wireless and wired networks. These risks stem from various malicious actions and security threats that have emerged with the development of Fourth Generation (4G), Fifth Generation (5G), and Internet of Things (IoT) networks. Machine learning (ML) algorithms have been applied to Intrusion Detection Systems (IDSs) due to their capacity to their ability to detect complex network traffic patterns. Deep learning (DL) networks are highly effective in processing images and videos and they have potential to solve other types of data. Given the characteristics of network traffic records used for intrusion detection in wireless and wired networks, we propose a simple data preprocessing method to convert the data into a grid-structured format, making it compatible with image processing networks. To validate the proposed structure, modified LeNet networks have been used for intrusion detection based on the NSL-KDD and CICIoV2024 (Canadian Institute for Cybersecurity Internet of Vehicles 2024 dataset) benchmark datasets. The simulation results indicate that methods based on extracted features may not always guarantee improved performance. The proposed Image Classification Neural Network-based Intrusion Detection (ICNN-ID) outperforms the compared existing methods. The multiclass classification experimental results show that the proposed LeNet-based IDS achieved a test accuracy (TAC) of 89.97% for NSL-KDD and nearly 100% (99.996%) for CICIoV2024. Additionally, it offers higher accuracy and improved robustness compared to a one-dimensional CNN and a recent deep learning model that integrates deep convolutional neural networks (DCNN) and bidirectional long short-term memory (BiLSTM).

* Corresponding author.

E-mail address: wangz@unisa.ac.za (Z. Wang).

<https://doi.org/10.1016/j.asoc.2025.113236>

Received 27 October 2024; Received in revised form 12 April 2025; Accepted 26 April 2025

Available online 13 May 2025

1568-4946/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

1. Introduction

The miniaturization of chips has promoted the rapid development of networks, including mobile networks, wireless networks and wired networks including computer networks. The network has greatly facilitated the acquisition of personal information and industrial upgrading. However, this advantage comes with increased vulnerability to various cyber threats. Intrusion Detection Systems (IDS) have become essential for safeguarding wireless networks by identifying and mitigating unauthorized access and malicious activities. Traditional IDS approaches often rely on signature-based detection methods, which, although effective against known threats, struggle with novel and evolving attacks. Artificial Intelligence (AI) refers to the simulation of human intelligence in machines, enabling them to perform tasks such as learning, reasoning, and problem-solving. Due to its power in various applications, the focus has shifted toward employing advanced AI techniques to enhance the efficacy of IDS [1], providing more robust and adaptive security solutions.

The incorporation of AI into IDS has revolutionized the field, enabling the detection of complex and sophisticated threats. Machine learning (ML) and deep learning (DL) algorithms, such as decision trees, support vector machines, and neural networks, are now widely utilized to analyze vast amounts of network traffic data, uncovering patterns indicative of cyber intrusions [2–4]. These AI-driven systems can adapt to new attack vectors, offering significant improvements in detection rates. The ongoing advancements in AI research continue to enhance the capabilities and effectiveness of IDS, making them indispensable in modern network security frameworks [5].

Compared to classical ML algorithms such as K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Random Forest (RF), and Naive Bayes (NB), deep learning algorithms typically achieve better performance [2] due to their automatic feature extraction, scalability, high representational power, adaptability, and ability to handle complex, high-dimensional data. This has attracted many researchers to focus on applying deep learning techniques in IDS development.

Deep learning algorithms can generally be divided into two categories: those without convolution operations and those with convolution operations. The algorithms without convolution, such as various types of recurrent neural networks (RNNs) including long short-term memory (LSTM), gated recurrent unit (GRU), and simple RNNs can directly take the features of each network connection record as input. However, they cannot effectively capture spatial or local patterns.

On the other hand, deep learning algorithms with convolution operations, such as LeNet [6], AlexNet [7], VGGNet [8], and GoogLeNet [9], have greatly advanced AI over recent decades. These models use convolution operations to abstract features from images and have demonstrated outstanding performance. While they were originally designed for image or video processing, convolution-based models have also shown potential in IDS applications.

Some IDS studies have applied convolution by either using one-dimensional (1D) convolutional layers [10] or by converting network traffic data into an image format with pixel values ranging from 0 to 255 [11]. However, 1D convolution may lack the capacity to fully capture local patterns, and converting network connection records into image formats may diminish the full potential of convolution operations, which are best suited for grid-structured data. Thus, it is desirable to develop a simple yet effective method to convert network connection records into a grid-structured format.

To improve the performance of ML-based IDS, several methods have been proposed, such as feature selection or extraction techniques, as demonstrated by [2–4] and their references. If the performance of the algorithms used for feature reduction is not perfect, the selected features may not be good enough to fully represent all the original features. This means that while the selected features can reduce computation cost, they might not guarantee improved performance, as there is a possibility of discarding some useful features. “What is rational is

real; and what is real is rational” [12]. If we use the full dimensional information, the features of a network connection/traffic record may be too numerous, leading to high algorithm complexity, especially when the inputs are data series. However, image processing networks can handle high-dimensional inputs. For example, for 128×128 images, the number of inputs is 16,384, which is manageable because convolutional networks are specifically designed to extract features and process data in a grid-like structure. This makes convolutional approaches particularly suitable for high-dimensional inputs if the data can be converted into grid-structured format.

Network connection feature datasets often suffer from class imbalance, where most samples are labeled as normal traffic, and the majority of cyberattack samples belong to a few common attack types. However, minority attacks, though infrequent, may pose higher risks to network security. To balance the dataset, several methods have been proposed, including Synthetic Minority Over-sampling Technique (SMOTE) [13], Adaptive Synthetic Sampling (ADASYN) [14], Random Over-Sampling (ROS) [15], and Random Under-Sampling (RUS) [16]. Nonetheless, these methods have their limitations. For example, SMOTE, ENN, and ADASYN may generate synthetic samples that do not accurately reflect real-world cyberattack behaviors; ROS can lead to overfitting and redundancy, increasing computational and storage requirements; RUS may discard critical data, potentially resulting in biased models. In this study, we propose a simple yet effective strategy: rearranging the training data to ensure all minority samples are used during training, without the need for data duplication or synthetic generation.

While CNNs are highly effective for grid-structured data like images, most network traffic data is non-sequential, time-independent, and discrete, making it inherently incompatible with standard CNN input formats. This mismatch has limited CNN application in IDS, particularly in terms of performance, generalizability, and computational efficiency. To address these issues, we explore how convolutional operations can still be leveraged by transforming network traffic data into a grid-structured format. This motivates the development of an image classification neural network-based IDS, referred to as ICNN-ID, which introduces a fixed-random data conversion method to bridge the structural gap. Additionally, we tackle the challenge of class imbalance by proposing a forced minority inclusion strategy. The main contributions of this paper are

- (1) We propose a novel intrusion detection framework based on image classification neural networks, which applies convolution operations to grid-structured representations of network traffic data. This flexible architecture can be extended to incorporate various ICNN-based models for enhanced detection.
- (2) We introduce a forced minority inclusion strategy to improve classification performance on imbalanced datasets. This method ensures that all available instances from minority classes are included in the training process when their occurrence falls below a predefined threshold, thus enhancing the model's exposure to rare attack types.
- (3) A new fixed-random data conversion technique is developed to reshape discrete, non-sequential network traffic data into a format compatible with ICNN processing. This technique randomly distributes features into a larger grid and reuses features where necessary, creating implicit spatial patterns that enrich feature diversity without altering the original data structure.
- (4) We demonstrate through case analysis that feature selection must be applied with caution in the context of intrusion detection. Our experiments show that, in some cases, using all features yields better results than using a reduced set selected by traditional methods like XGBoost.
- (5) We conduct extensive experimental evaluations using benchmark datasets (NSL-KDD and CICIoV2024), comparing ICNN-ID against both conventional and state-of-the-art models. Performance is measured using multiple metrics (Accuracy, Precision,

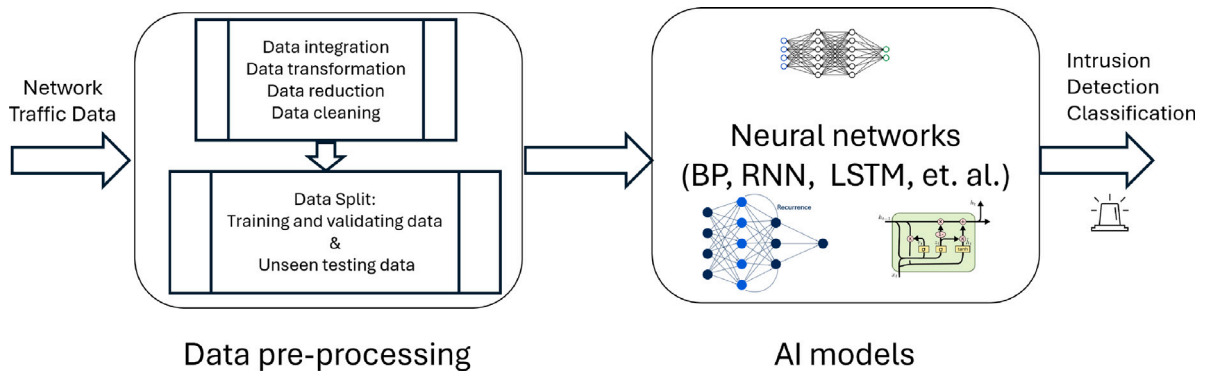


Fig. 1. Neural network based IDS.

Recall, F1-score), and statistical significance tests (e.g., t-tests) are applied to validate the robustness of our findings.

The remaining sections of this paper are as follows: in Section 2, we present a brief literature review of some concepts such as intrusion detection systems and image processing networks; In Section 3 we describe the new methodology for intrusion detection including the system architecture, the flow charts and important operations; in Section 4, we present and analyze the experimental results, and Section 5 gives the conclusion of our research and give some ideas for future works.

2. Related work

2.1. Intrusion detection system based on neural networks

Intrusion Detection Systems have become increasingly vital in securing modern wireless and wired networks, which are inherently susceptible to a wide range of cyber threats due to their open and distributed characteristics. With the rise of sophisticated attacks, traditional signature-based IDS methods have proven inadequate [2]. This has led to the integration of AI, specifically neural networks, into IDS to enhance their ability to detect and mitigate such complex threats. The general structure of neural networks based IDS is shown in Fig. 1. As can be seen from this figure, the IDS algorithms include two main parts data pre-processing and AI models. In the field of traditional machine learning, there is a popular saying: “**The data and features determine the upper (performance) limit of machine learning, while the model and algorithms merely help to approach this limit.**” Data preprocessing plays a key role in helping models and algorithms achieve this limit and also determines which types of models can be used to process the data and fulfill the desired function.

The integration of neural networks into Intrusion Detection Systems (IDS) has demonstrated significant potential due to their capability to model complex patterns and learn from extensive datasets. Various neural network architectures, such as Backpropagation (BP) Neural Networks [17], Recurrent Neural Networks (RNNs), and hybrid models, have been effectively employed in detecting wireless intrusions. RNNs and their variants, including Long Short-Term Memory (LSTM) networks, are particularly adept at identifying sequential patterns in data, making them suitable for detecting attacks that unfold over time, such as Distributed Denial of Service (DDoS) attacks. Studies have demonstrated that LSTMs can effectively model the temporal dependencies in network traffic data, thereby improving the detection accuracy of IDS [18].

Combining different neural network models and integrating classical machine learning algorithms have also been explored to create more robust IDS [3,19,20]. For instance, a hybrid model combining LSTM and Gated Recurrent Unit (GRU) networks was proposed, where the outputs of LSTM and GRU are merged before entering

a fully connected network [20]. While such ensemble methods can improve performance, they introduce increased complexity and computational overhead. Moreover, although these AI models effectively process network traffic data, they often struggle to efficiently capture local patterns and inherent spatial hierarchies within individual samples.

Recent research in intrusion detection has explored several advanced methodologies in combination with deep learning algorithms. Explainable AI (XAI) has been introduced to enhance the transparency of IDS models, providing interpretable decision-making processes that improve trust in cybersecurity applications [21–23]. In one study, techniques such as self-attention mechanisms, adaptive fusion of multi-scale data, context broadcasting mechanisms, and residual connections were employed for neighboring edge aggregation, resulting in a more complex model with higher computational costs [24]. Additionally, Graph Deep Learning (GDL) and Generative Adversarial Networks (GANs) have been utilized in IDS for their ability to extract complex feature representations and improve anomaly detection [25,26]. However, compared to RNNs, LSTMs, and GRUs, GDLs are harder to train, require graph-structured data, are less scalable, and suffer from over-smoothing in deep layers. GANs, on the other hand, face challenges such as training instability, mode collapse, and difficulty in balancing the generator and discriminator, which can hinder their effectiveness in IDS applications. Furthermore, optimization algorithms have been integrated into IDS models to improve detection efficiency and accuracy, allowing for better adaptability in real-world scenarios [27]. While these approaches offer significant performance improvements, they often come with higher computational costs, which can be a limiting factor for resource-constrained applications.

In addition to the studies reviewed in this work, there are comprehensive review papers on neural network-based IDSs that readers can refer to for more in-depth information [1,28,29].

2.2. Intrusion detection using CNNs

Convolutional Neural Networks (CNNs) are a type of neural network that have brought significant advancements to image processing, especially in tasks like image classification, object detection, and segmentation. CNNs are specifically optimized for handling grid-structured data such as images, videos, and audio spectrograms, making them highly effective for tasks involving large input data sizes. Their architecture, inspired by the human visual cortex, is particularly adept at processing high-dimensional spatial data, and CNNs can capture local patterns and hierarchical structures within the input, making them powerful for visual recognition tasks. In CNNs, the convolution operation is a fundamental process used to extract features from input data as shown in Fig. 2, typically images. It works by sliding a small matrix, called a kernel or filter, over the input data. This kernel performs element-wise multiplication with the corresponding section of the input and then sums the results to produce a single value, which forms part of the output, known as a feature map.

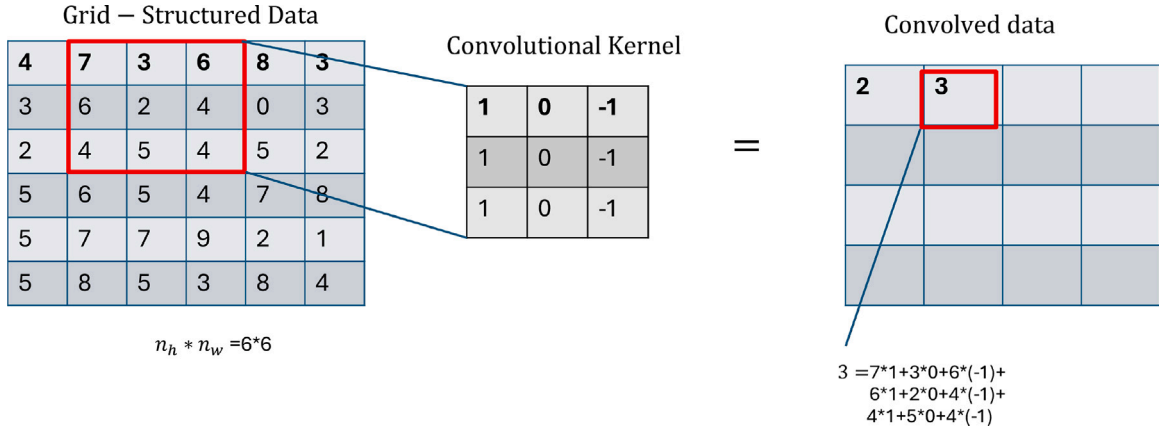


Fig. 2. Convolution operation.

This operation helps CNNs capture local patterns such as edges, textures, and shapes in images, enabling the network to recognize and learn hierarchical features as it moves through deeper layers. The convolution operation preserves spatial relationships in data, making it highly effective for tasks like image classification, object detection, and segmentation.

CNNs are built from multiple layers designed to automatically learn feature representations from input images. Besides convolutional layers, these layers include pooling layers, which reduce the dimensionality of the data for greater computational efficiency and robustness to small shifts in the image; and fully connected layers, which integrate learned features to make final predictions. One of CNNs' core strengths is their ability to build hierarchical feature representations. While the earlier layers capture basic visual elements such as edges, the deeper layers are capable of recognizing more complex patterns, including objects or faces. This multi-level feature extraction allows CNNs to excel in tasks like image classification with high accuracy. Given the powerful capabilities of CNNs, several studies have explored their application in IDS. Before introducing the CNN based IDS, we give a brief introduction about the network traffic data.

Network traffic data can be complex, with numerous features (or dimensions), such as packet size, duration, protocol type, source/destination IP, port numbers, and so on. To better understand the network data, we give a brief introduction of one famous dataset **NSL-KDD**, a refined version of the KDD Cup 1999 dataset [30]. NSL-KDD addresses the issues of redundancy and imbalance by removing duplicate records, ensuring a more equitable representation of both normal and attack traffic. NSL-KDD provides distinct training and testing subsets, making it easier to assess the performance of machine learning models without bias. This dataset is widely used in IDS research for training machine learning models to detect network intrusions.

The NSL-KDD dataset contains **41 features** for each network connection record. These features are categorized into the following four groups:

(1) Basic Features: These are attributes that can be derived from the packet headers without inspecting the payload. *Example features:* duration, protocol_type, service, src_bytes, dst_bytes.

(2) Content Features: These features are based on the payload of the packets, inspecting the data content within a connection. *Example features:* count, error_rate, srv_error_rate, dst_host_srv_count.

(3) Time-based Traffic Features: These are derived by observing the connections over a specific time window to detect patterns over time. *Example features:* same_srv_rate, diff_srv_rate, dst_host_count, srv_count.

(4) Host-based Traffic Features: These features inspect the network connections related to a specific host and look for patterns across those

connections. *Example features:* dst_host_srv_diff_host_rate, dst_host_error_rate, dst_host_rerror_rate.

In addition to the 41 features, each record in the dataset also has a label indicating whether the connection is normal or an attack, and if it is an attack, the type of attack. For more details, please refer to [30].

To effectively apply CNNs to IDS, network traffic data must be transformed into formats compatible with CNN architectures. A straightforward approach involves utilizing 1D convolution operations on sequences of preprocessed network features, such as those obtained through encoding and normalization. These features can be arranged as a single row vector, serving as input to 1D CNNs, as demonstrated in studies like [10,31]. While 1D CNNs are efficient for sequential data and can detect local patterns effectively, they may not fully exploit the spatial dependencies present in data with multiple dimensions, potentially affecting the IDS's performance in tasks requiring the understanding of such intricate relationships.

Beyond 1D CNNs, higher-dimensional convolution operations have been explored for IDS applications. For instance, [11] proposed converting internet traffic data into image formats by mapping feature values to pixel intensities using functions like `np.asarray()`, followed by processing with CNNs. Similarly, DeRose et al. [19] introduced a color imagery encoding technique based on chunking. However, such pixel value conversions may alter the semantic meaning of network traffic data, potentially distorting patterns and affecting classification accuracy. Zhu et al. [32] employed a feature tokenizer method to convert network communication data into two-dimensional grid-structured formats, enabling CNN-based intrusion classification. While effective, this approach can be computationally intensive, especially when dealing with high-cardinality features or limited datasets.

To enhance performance, some IDS frameworks integrate CNNs with other artificial intelligence models. For example, Bamber et al. proposed a hybrid CNN-LSTM approach [33], combining 1D CNNs with Long Short-Term Memory networks. Xue et al. [34] utilized a fully connected layer to transform feature vectors into two-dimensional data, which are then processed by a CNN-based hybrid classification network. However, these data conversion processes can be computationally expensive and susceptible to noise, potentially impacting the robustness and efficiency of the IDS.

In summary, while CNNs offer powerful capabilities for feature extraction in IDS, the methods of data transformation and integration with other models must be carefully considered to balance performance gains against computational costs and potential data distortion.

2.3. Feature extraction based IDS

In intrusion detection, datasets often contain a large number of features, some of which may be redundant, irrelevant, or even noisy.

Feature selection of IDS involves identifying the most relevant and informative features from network traffic data to improve detection accuracy, reduce computational complexity, and minimize overfitting. Feature selection techniques in IDS are broadly categorized into three types [28]:

(1) Filter methods: These methods evaluate the relevance of features independently of any learning algorithm. Common techniques include information gain, chi-square test, correlation-based selection, and mutual information. Filter methods are fast and scalable but may overlook interactions between features.

(2) Wrapper Methods: Wrapper methods evaluate subsets of features by training and testing a model on different combinations. Algorithms like recursive feature elimination and forward/backward selection fall into this category. While often more accurate, wrapper methods are computationally expensive and less scalable.

(3) Embedded Methods: Embedded methods perform feature selection during the model training process. Examples include LASSO (L1 regularization), decision tree-based methods (e.g., random forest importance), and gradient boosting (e.g., XGBoost). In summary, embedded methods may struggle to capture complex feature interactions, can introduce bias toward certain feature types, and may become computationally expensive on large datasets. Additionally, without proper regularization, they risk overfitting and reduced generalizability.

Recent advancements also explore hybrid approaches that combine multiple selection strategies for improved robustness and performance. For example, combining Principal Component Analysis (PCA) and Random Forest (RF) has been shown to enhance performance [35]. Additionally, evolutionary algorithms [36] and deep learning-based attention mechanisms [28] are gaining traction for automatic feature selection. However, these hybrid approaches often incur higher computational costs. For a comprehensive overview of feature selection techniques, readers can refer to [28] and its references.

A feature extraction-based IDS can reduce computational costs, as fewer features are used to train the model and make predictions. However, it may not improve performance since the baseline function used to extract features does not guarantee that the chosen features are the most important ones. In this subsection, we aim to investigate whether a feature extraction-based IDS, leveraging an AI model, can achieve a significant improvement in performance. Specifically, a multi-layer perceptron (MLP), a type of artificial neural network commonly used for classification tasks, is applied to the NSL-KDD dataset to evaluate the performance of an IDS using different feature extraction methods.

Similar to [3], the NSL-KDD dataset is divided into training, validation, and test sets: the training and validation sets are drawn from the KDDTrain dataset with a 75:25 ratio, while KDDTest+ is used as the test set. Each record in the NSL-KDD dataset contains 41 features, as shown in Table 1. Various methods can be used to extract features; for example, the XGBoost feature selection approach ranks the importance of features (also called attributes), allowing the selection of the most important ones before applying AI models for intrusion classification. Referring to [3], twenty-two (22) features have been selected: f5, f29, f38, f35, f37, f23, f2, f30, f6, f33, f25, f8, f13, f3, f36, f10, f17, f40, f32, f12, f34, and f4. In addition to the features selected using XGBoost, we simply selected the first 22 features: f1 through f22, to illustrate that feature selection does not always guarantee performance improvement, and in cases where the effectiveness of selected features is uncertain, using all features may serve as a viable alternative. Three cases are considered in this study: (1) all features; (2) 22 features selected by XGBoost; and (3) the first 22 features. The rule for selecting the feature importance threshold in the XGBoost-based feature selection method is given in [3] by

$$\text{if } FI_{\text{next}} < \eta \times FI_{\text{current}}, \text{ then } FI_{\text{current}} = FI_{th}$$

where FI_{current} is the current feature importance value; FI_{next} is the next feature importance value in the sorted list; FI_{th} is the feature importance threshold used for feature selection; η is a tuning parameter,

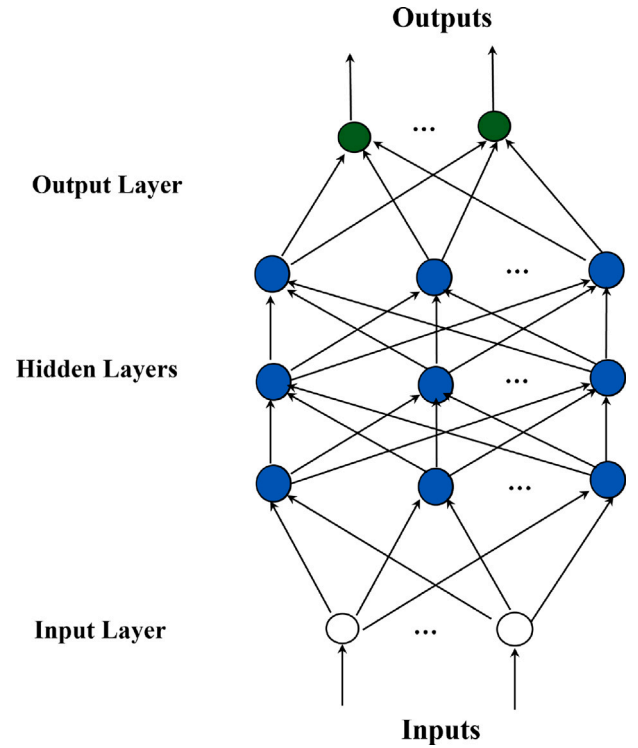


Fig. 3. Representative diagram of MLP.

empirically set to 0.35. This rule ensures that features with a significant importance drop between consecutive rankings are excluded from the final subset.

To simplify the comparison, this section focuses solely on binary classification, categorizing network traffic or system behavior into two distinct classes: normal (benign) and malicious (attack).

The neural network classifier is an MLP model composed of three hidden layers, each containing 30 neurons. Fig. 3 gives a representative diagram of the Multi-Layer Perceptron (MLP) model. The MLP model consists of a fully connected neural network with an input layer, three hidden layers, and an output layer. This architecture allows the model to learn complex patterns in the data by passing information through multiple layers of neurons that progressively capture higher-level representations. The model uses a feedforward structure, where the input data is passed through the hidden layers and finally to the output layer, where predictions are made. During training, the model adjusts its internal weights through backpropagation, which minimizes the error between the predicted and actual outputs. The learning process involves updating the weights in the direction that reduces the classification error, allowing the network to become better at classifying new, unseen data. The overall structure of this neural network is designed to handle complex classification tasks by effectively capturing non-linear relationships in the data.

The simulation results, presented in Table 2, demonstrate that the IDS using features selected by XGBoost performed the worst compared to the IDS using all features and the IDS based on the first 22 features. The results also highlight that using fewer features can reduce the computational cost of AI models. From this comparison, we can conclude that the feature extraction method significantly impacts the IDS performance, and features deemed highly important by selection algorithms may not be as effective for attack classification when using different AI models. For the following ICNN-ID experiment, we will use all available features without applying feature selection to evaluate the model's capability using the complete feature set, ensuring that no potentially important features were excluded.

Table 1
NSL-KDD attributes description.

No.	Attribute	Format	No.	Attribute	Format
f1	duration	numeric	f22	is_guest_login	numeric
f2	protocol_type	categorical	f23	count	numeric
f3	service	categorical	f24	srv_count	numeric
f4	flag	categorical	f25	error_rate	numeric
f5	src_bytes	numeric	f26	srv_error_rate	numeric
f6	dst_bytes	numeric	f27	error_rate	numeric
f7	land	numeric	f28	srv_error_rate	numeric
f8	wrong_fragment	numeric	f29	same_srv_rate	numeric
f9	urgent	numeric	f30	diff_srv_rate	numeric
f10	hot	numeric	f31	srv_diff_host_rate	numeric
f11	num_failed_logins	numeric	f32	dst_host_count	numeric
f12	logged_in	numeric	f33	dst_host_srv_count	numeric
f13	num_compromised	numeric	f34	dst_host_same_srv_rate	numeric
f14	root_shell	numeric	f35	dst_host_diff_srv_rate	numeric
f15	su_attempted	numeric	f36	dst_host_same_src_port_rate	numeric
f16	num_root	numeric	f37	dst_host_srv_diff_host_rate	numeric
f17	num_file_creations	numeric	f38	dst_host_error_rate	numeric
f18	num_shells	numeric	f39	dst_host_srv_error_rate	numeric
f19	num_access_files	numeric	f40	dst_host_error_rate	numeric
f20	num_outbound_cmds	numeric	f41	dst_host_srv_error_rate	numeric
f21	is_host_login	numeric			

Table 2
Comparison of IDS with feature extraction.

Cases	Validation accuracy	Test accuracy	Training time (s)
All features	99.82%	81.03%	41.83
XGBoost features	99.80%	81.94%	38.40
First 22 features	99.21%	83.90%	30.37

Although the classical feature selection technique, XGBoost, as demonstrated in our simulations, did not consistently improve IDS performance, some advanced feature selection methods, such as Tensor-Based Error Robust Unbalanced Incomplete Multi-view Unsupervised Feature Selection (TERUIMUFS) [37] and Structural Regularization-Based Discriminative Multi-View Unsupervised Feature Selection (SDFS) [38], offer promising alternatives. TERUIMUFS addresses challenges associated with unbalanced and incomplete multi-view datasets by leveraging tensor decomposition techniques to capture the underlying data structure. This method enhances the robustness of feature selection against errors and missing data, thereby improving the performance of models trained on such datasets. On the other hand, SDFS selects a compact and representative feature subset from multi-view data by computing similarity matrices for each view and learning a consensus similarity graph. This approach preserves the geometrical structure of the data without introducing additional parameters, thereby enhancing the discriminative ability of the selected features. This study focuses on using all features, and feature selection was not applied to ICNN-ID, although these techniques could potentially be integrated into ICNN-ID for further optimization.

3. Image classification neural network based intrusion detection method

A significant amount of effort has been invested in CV using AI, and many excellent AI models have been proposed. To fully leverage the power of image classification networks, we must preprocess the data to align it with the input structure of these networks. Convolution is a commonly used operation in AI models, such as U-Net, ResNet, and YOLO [39], for processing images, videos, or grid-data. Therefore, we chose a convolution-based network as the baseline to propose a new IDS structure based on an image classification neural network, as shown in Fig. 4. In this model, any image classification network, such as LeNet or AlexNet, can be used with or without minor revision, and the network traffic data needs to be transformed into a suitable structure before entering the classification networks. To adapt image classification networks for IDS and consider the characteristics of network

traffic data, we propose an intrusion detection method based on image processing networks. This approach demonstrates that AI models used for image classification can also be applied to intrusion detection. The paper focuses on the data preprocessing shown in Fig. 4, which involves two key components: data balancing and data conversion, discussed in the following two subsections; thereafter a brief description about image classification network is provided in the third subsection.

3.1. Forced minority inclusion method

Data balancing refers to the techniques used to handle datasets where certain classes are significantly underrepresented. In datasets like NSL-KDD, where attack types like spy, perl, and phf have only 2, 3, and 4 samples respectively (out of over 12,000 total samples), this imbalance can severely affect the model's ability to correctly identify these rare attacks. Common strategies for addressing this include over-sampling the minority class, undersampling the majority class, or using techniques like SMOTE (Synthetic Minority Over-sampling Technique) to artificially generate more samples of the underrepresented class. For the wireless network traffic features, each record is unique and it is difficult to use some data balance methods. According to our investigation, one factor affecting IDS performance is that some attacks with very few samples may not be included in the training set and are instead assigned to the validation set. Here, a Forced Minority Inclusion (FMI) method is proposed before splitting the training set and validation set whose flow chart is shown in Fig. 5. Here, i is the index for counting processed samples (or a loop counter up to N_{total}) and j is the index of the current label category (or a loop counter up to N_{labels}). The flowchart in Fig. 5 illustrates FMI, which ensures that minority-class samples are adequately represented in the training set. This method is particularly useful for handling imbalanced datasets by enforcing the inclusion of underrepresented categories.

In this method, there is only one parameter should be set and it is the threshold P_{thr} which determines which label categories are all moved to the training set. For example, if the number of samples of one specific label category is less than $P_{thr}\%$ like 1% (or other percentages), all the samples of this label category are included to the training set.

While the FMI effectively addresses class imbalance, it may also amplify issues related to outliers and duplicate samples within the minority class. If not properly handled, these factors could lead to erroneous pattern learning or overfitting to specific samples. Therefore, ensuring the quality of minority class samples through appropriate data preprocessing is essential before applying the FMI Method. To mitigate the impact of outliers, we recommend using statistical methods such as

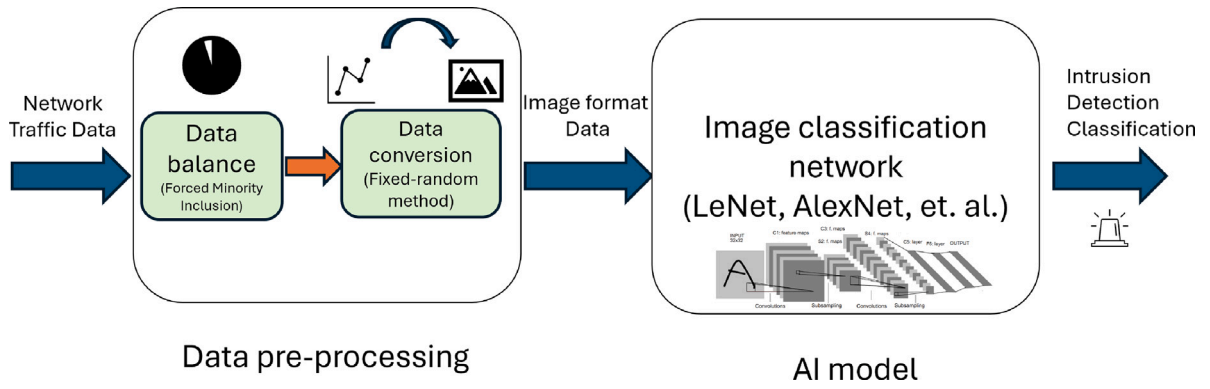


Fig. 4. General IDS using image classification neural network.

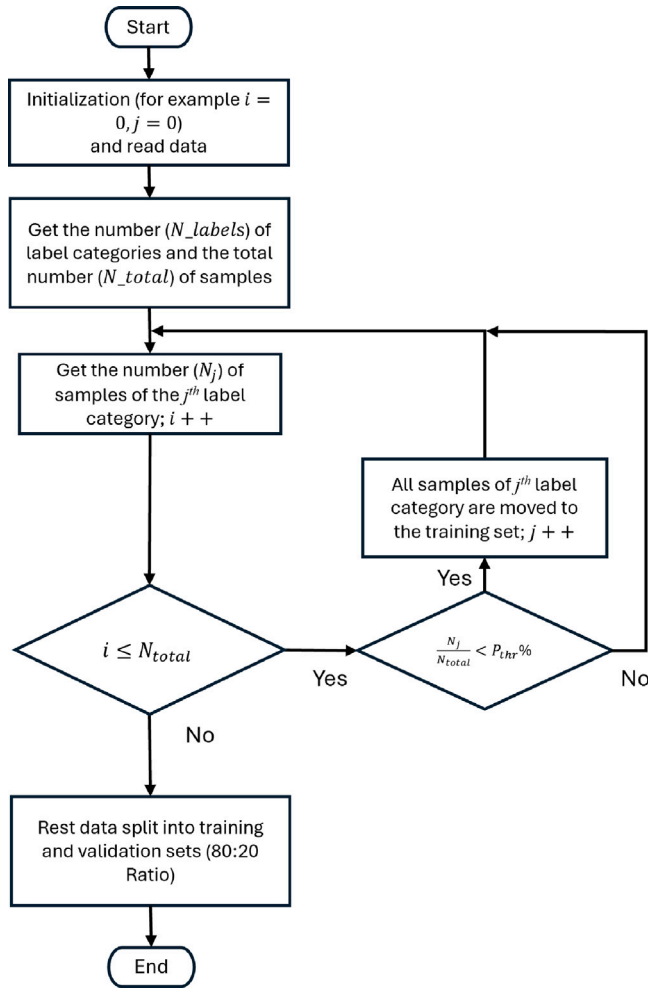


Fig. 5. Forced minority inclusion method.

appear very infrequently. In such cases, strict outlier detection may lead to an excessive reduction of already limited samples, negatively impacting the model's ability to learn from these categories. Given this data scarcity, our study assumes that the dataset samples are reasonably representative and does not apply additional filtering to these rare categories.

3.2. Fixed-random data conversion method

Before using an image classification neural network, especially CNN, we need to convert the network traffic data into a format compatible with the image classification network, as shown in Fig. 4. According to the features of each network connection record described in Section 2.2, they are non-sequential data with the following characteristics: (1) **Independence**: Each feature is generally independent, with no strong dependencies between them (for example, there is no correlation between count and error_rate); (2) **No time dependency**: Unlike time-series data such as video or audio, network traffic features do not follow a sequential time order and can be shuffled without affecting their representativeness; (3) **Discreteness**: Each feature sample (e.g., duration) is discrete and not continuous, meaning there is no natural connection from one feature to another.

The convolution operation is a fundamental mathematical process used primarily in image and signal processing, where a filter or kernel slides over the input data (e.g., an image or a sequence) and performs an element-wise multiplication followed by summation. This operation focuses on localized patterns by applying the same filter across the entire input, which captures local features like edges or textures in images. The key characteristics include local connectivity, parameter sharing, and reduction of computational complexity by focusing only on relevant local information in the data.

Due to the lack of time dependency in network traffic data, the 1D convolution operation [10,31] may not fully capture the features of IDS data. To use convolution operations, commonly applied in image classification networks, it is necessary to convert non-sequential data into grid-structured data. Ref. [11] proposed using tabular transformation with the Python function np.asarray() to convert data into images. Similarly, [19] transformed non-sequential data into images using color imagery encoding combined with tabular transformation. Convolution uses filters (kernels) to scan the data in patches, capturing local spatial features. However, network connection records exhibit properties of independence and discreteness. Considering the characteristics of non-sequential data and the performance achieved using convolution operations, there is potential to further improve results in image processing networks. Convolution operations can help uncover features in non-sequential data, provided the data is appropriately pre-processed to be compatible with convolution. By transforming non-sequential data into a suitable format for CNN-based processing, deeper insights can be uncovered, thereby enhancing overall detection performance.

the Interquartile Range (IQR) method and Z-score analysis to detect and either remove or adjust extreme values. For duplicate samples, employing similarity-based deduplication using cosine similarity measures or hashing algorithms can help detect and either eliminate or reduce the weight of repeated instances. However, these techniques are more suitable for datasets with a moderate number of samples and may be computationally expensive for extremely large datasets. In datasets such as NSL-KDD, certain attack types are inherently rare. For example, attack types such as spy (2), perl (3), phf (4), multihop (7), ftp_write (8), loadmodule (9), rootkit (10), and imap (11)

Our primary objective was to develop a CNN-based intrusion detection system capable of effectively processing time-independent and discrete network traffic data. Traditional CNNs excel with grid-structured inputs, such as images; however, network traffic data does not naturally conform to this format. To bridge this gap, we introduced the fixed-random data conversion technique, which transforms discrete network data into a grid-structured format suitable for CNN processing.

Based on the properties of the convolution operation, feature extraction is not limited to image data but can be applied to any grid-structured data. Consequently, using color image encoding, which may obscure or lose important features, is not always necessary. Additionally, network traffic data does not need to conform to specific image properties, such as pixel count. In this study, we propose a simple and direct data formatting method to convert non-sequential data into a grid structure without being constrained by the number of features. The corresponding flowchart is shown in Fig. 6, which illustrates the process of converting network traffic data into grid-structured data.

Here, there are two parameters, w and h , which define the structure of the required grid-structured data as determined by the input to the image-processing network. N_{total_sample} refers to the number of samples including training, validation, and test sets. In this study, we only consider 2D grid structure, which can be easily extended to higher-dimensional grid-structured data by adding more parameters.

One of the most important steps involves data extension, as mentioned in Fig. 6, which is different from existing methods. Since the network connection records or features are independence, with no time dependency and discreteness, there is no specific rule to follow for arranging the relationship of the features in a record viewed as a vector. As shown by the flowchart, we first define a variable FR (Fixed-Random vector), which contains $w \times h$ integers from 0 to $w \times h - 1$, representing the indexes of the features in a network connection record. Here, we assume that the number of features in a network connection record is less than $w \times h$. Of course if it exceeds $w \times h$, we can apply a feature extraction method to reduce the number of features in each record.

Once FR is defined, its members are shuffled. Next, we extend the features to ensure its number equals $w \times h$. There are various methods to achieve this, and the simplest one is to copy the features cyclically until the number of features reaches $w \times h$. Third, we use the shuffled FR as the indices for the extended network record to rearrange/reshuffle the features. Finally, a function like `reshape()` is used to convert the extended and shuffled features of a record to the specific grid-structure compatible with the image processing network.

For example, given a 3×3 grid (9 elements) and a feature vector $A = [f_1, f_2, f_3, f_4]$ containing only four features, the extended vector B can be constructed as: $B = [f_1, f_2, f_3, f_4, f_1, f_2, f_3, f_4, f_1]$. To introduce variability and prevent biases, the elements of B are mapped to a shuffled version (C) using a predefined index set FR . The FR vector is generated in two steps: (1) defining FR as a sequence of integers (0 to 8) representing the indexes of B , and (2) shuffling FR and fixing its order throughout training, testing and implementation. The final grid-structured representation D is obtained by reshaping C into the predefined 3×3 grid.

This method is called the Fixed-Random Data Conversion Method because the shuffled FR remains unchanged once determined for a particular application, ensuring that the transformation process remains deterministic across different runs. Although the method involves an initial random shuffling of indices, this randomness is fixed and consistent throughout training and inference, preventing any variations in the data transformation process across multiple executions of the model. This guarantees reproducibility and stability in performance.

Here, we are using the name “fixed-random” since the “fixed-random” aspect ensures that the transformed data follows a random distribution, thereby avoiding biases associated with fixed data structures. This randomness contributes to stable detection performance by preventing the model from overfitting to specific patterns inherent in a non-random structure. Additionally, in this technique, some features

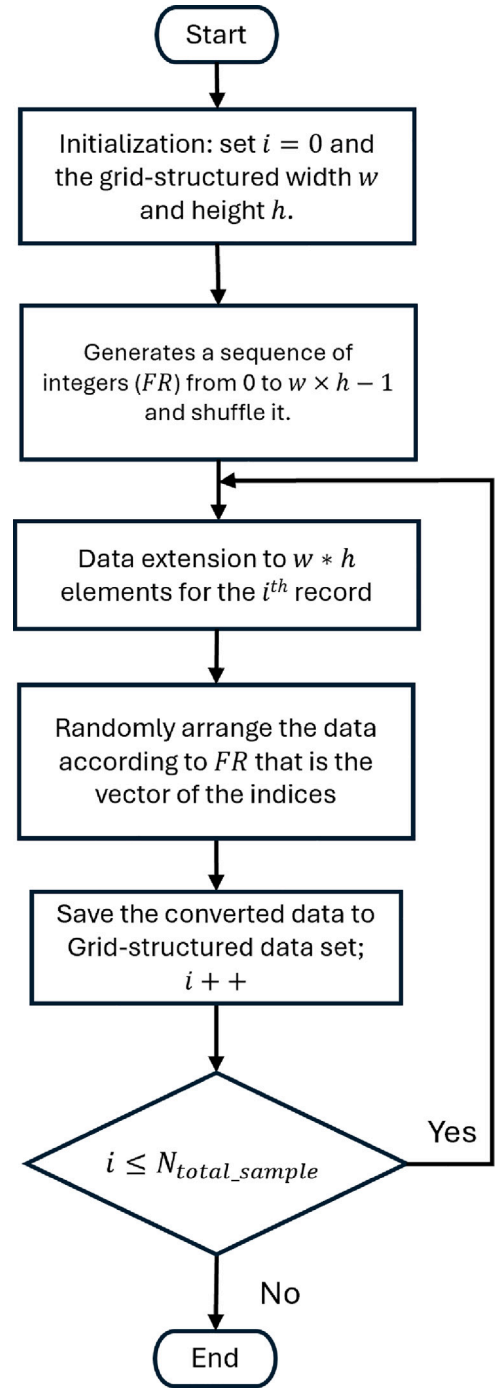


Fig. 6. Fixed-random data conversion method.

within a record are randomly arranged within the grid structure. Since the grid contains more elements than the number of features in a discrete network connection record, certain features are reused to fill the grid, leading to the construction of spatial features. This transformation can be seen as an implicit form of data augmentation, as it increases variability in the input representation while preserving the integrity of the original feature set.

3.3. Image classification network

Most image classification networks are processing the images with three Color Channels: Each pixel in an RGB image is represented by

three values corresponding to Red, Green, and Blue intensities. For example, in CNNs, RGB layers are processed together by convolutional filters. Each filter processes all three channels (Red, Green, and Blue) simultaneously, rather than treating them separately. Applying image classification networks directly to network security data can result in inefficient processing resource utilization. This is particularly true when using RGB-based architectures, which are optimized for natural image processing but introduce unnecessary computational overhead when applied to grid-structured network data. To address this concern, we propose using a single-channel network instead of a three-channel RGB-based network, thereby reducing computational costs while preserving classification performance. If we maintain the RGB-based network, we can convert data to a range between 0 and 255 and create three channels by copying the same data.

In the domain of image-based neural networks, recent studies have proposed advanced techniques to enhance classification accuracy. The Meta-Ensemble Explainable AI Approach has been introduced for AI-synthesized image classification, leveraging ensemble learning to improve interpretability and robustness [40]. Similarly, a Graph Convolutional Processing Framework has been developed for medical image classification, demonstrating improvements in feature extraction and classification performance [41]. Moreover, optimization algorithms have been applied to image classification tasks to enhance model efficiency, ensuring improved feature selection and learning processes [42]. While these new methods provide significant advancements, they often require extensive computational resources, making them less feasible for real-time IDS applications. In this study, we focus on a computationally efficient approach, utilizing basic CNN architectures and their variants to achieve reliable intrusion detection with manageable computational costs.

4. Experimental results and analyses

To validate ICNN-ID, we choose two famous data sets to test the proposed methods and compare its performance with some existing IDSs. The chosen datasets include NSL-KDD and CICIoV2024. The NSL-KDD dataset is an improved version of the widely used KDD'99 dataset, designed to enhance the evaluation of IDS. Though smaller and more efficient than its predecessor, NSL-KDD still maintains the complexity needed for rigorous IDS research, making it a reliable resource for developing and benchmarking modern network security algorithms. One of its advantages is that it includes a separate, unseen test set, which allows for an objective evaluation of model generalization. Additionally, many previous studies have utilized NSL-KDD, making it easier to compare the performance of our proposed model (ICNN-ID) with existing methods. The features of NSL-KDD have been introduced in Section 2.2.

The CIC IoV Dataset 2024 called CICIoV2024, provided by the Canadian Institute for Cybersecurity, is an extensive data repository designed to aid the development and evaluation of security solutions in the Internet of Vehicles (IoV) [43]. The CICIoV2024 dataset is derived from extensive real-world experiments conducted on the ECUs of a 2019 Ford vehicle, providing a detailed view of intra-vehicular communications. This dataset serves as a benchmark to advance cybersecurity solutions in the Internet of Vehicles (IoV), making it highly relevant to modern automotive cybersecurity challenges. This dataset encompasses a wide range of simulated IoV environments to reflect various real-world scenarios that connected vehicles might encounter. These simulations include diverse traffic conditions, communication scenarios between vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I), and multiple cybersecurity threat models, such as false data injection, denial of service (DoS) attacks, and spoofing attacks. Here, we have chosen the CICIoV2024 dataset to test the performance of IDS since electric vehicles are becoming increasingly popular due to their role in reducing greenhouse gas emissions. Moreover, intrusions into vehicle systems can have fatal impacts on life and property.

The simulations are conducted on a computer running Python 3.11 with the necessary packages, an Intel(R) Core(TM) i7-8700 CPU @ 3.20 GHz, 16 GB of RAM, an NVIDIA GeForce GTX 1060 6 GB and the Windows 11 operating system.

4.1. Performance metrics

Before introducing the performance metrics, the following variables are defined: TP (True Positives): Correctly identified positive cases.

FP (False Positives): Incorrectly identified as positive cases.

TN (True Negatives): Correctly identified negative cases.

FN (False Negatives): Incorrectly identified as negative cases.

The following metrics are used to evaluate the performance of classification models:

(1) Precision

$$\text{Precision} = \frac{TP}{TP + FP},$$

which measures the proportion of correct positive predictions.

(2) Recall

$$\text{Recall} = \frac{TP}{TP + FN},$$

which measures the proportion of actual positives that are correctly identified.

(3) Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

which measures the overall correctness of predictions.

(4) F1-Score

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

It is the harmonic mean of Precision and Recall.

(5) Confusion Matrix

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

The confusion matrix shown above is specifically for binary classification. For multiclass classification, the confusion matrix would be larger ($n \times n$), where n is the number of classes, and each cell would represent the count of predictions for each true/predicted class combination.

4.2. Experiments on ICNN-ID using LeNet

In this section, we present our experiments applying ICNN-ID. For generalization, we use the LeNet architecture as the baseline image classification network because of its computational efficiency. We evaluate intrusion detection for both binary and multiclass classification tasks, utilizing the NSL-KDD and CICIoV2024. Here, the size of the grid structure is 28×28 (784-element grid). The choice of 28×28 is made for consistency with commonly used CNN architectures, such as LeNet and EfficientNet-B0, which are optimized for image input sizes of 28×28 or 32×32 . Since the features are randomly distributed within the grid, their arrangement follows a uniform distribution, ensuring that the model's performance remains stable, as confirmed by our simulation results. Regarding alternative grid sizes, increasing the grid size would increase the computational cost due to a larger input space. However, because the data is shuffled and distributed uniformly, different configurations (e.g., non-square grids) would not significantly impact model performance. In this section, we use accuracy and the confusion matrix as performance metrics.

Hyperparameter selection is a crucial aspect of optimizing the performance of AI models, including LeNet. In general, there are several widely adopted strategies for choosing hyperparameters: (1) Empirical selection through trial-and-error, where different configurations

Table 3
NSL-KDD binary classification.

Cases	Validation accuracy	Test accuracy
All features	99.46%	90.75%
XGBoost features	99.40%	87.35%
First 22 features	98.57%	86.07%

are tested iteratively; (2) Utilizing hyperparameters reported in prior research, as these values have been proven effective in similar contexts; and (3) Employing optimization algorithms, such as grid search, random search, or Bayesian optimization, to systematically explore sub-optimal or optimal hyperparameter configurations. In our study, we used the hyperparameter settings reported in previously published work [44], as they have demonstrated efficiency and stability in similar model architectures. This approach ensures reproducibility and allows us to focus on assessing the impact of our proposed modifications rather than extensively tuning hyperparameters. This model consists of two convolutional layers followed by three fully connected layers. The first convolutional layer applies six 5×5 filters to the input, followed by a max-pooling layer (pool) that reduces the spatial dimensions. The second convolutional layer takes the six feature maps from the previous layer and applies sixteen 5×5 filters, followed by another max-pooling operation. The output is flattened and fed into a fully connected layer with 120 units, then passed through another fully connected layer with 84 units. The final output is passed through the third fully connected layer, which produces 2 output units, suitable for binary classification tasks. Each convolutional and fully connected layer is followed by a ReLU activation function to introduce non-linearity. The input of the model is a grid-structured data with a size of 28×28 .

4.2.1. NSL-KDD

Firstly we applied ICNN-ID to the binary classification of NSL-KDD and the results are shown in Table 3. It should be noted that the validation accuracy differs from the test accuracy because the validation set is split from the training set file KDDTrain+.txt, while the test set is derived from KDDTest+.txt. The KDDTest+.txt and KDDTrain+.txt files are part of the NSL-KDD dataset, an improved version of the original KDD Cup '99 dataset used for evaluating network intrusion detection systems. KDDTrain+.txt often has a different distribution of classes (i.e., types of attacks and normal interactions) compared to KDDTest+.txt. This discrepancy can include a different proportion of normal versus attack records as well as the presence of specific types of attacks not well represented or completely absent in the training data. Hence, the performance of the same algorithm on the validation set split from KDDTrain+.txt differs from the performance on the test set (KDDTest+.txt).

To further investigate the performance of ICNN-ID, we apply it to the multiclass classification for NSL-KDD dataset. The NSL-KDD dataset contains five main categories of network traffic: **normal**, **Denial of Service (DoS)**, **Probe**, **Remote to Local (R2L)**, and **User to Root (U2R)**. **Normal** represents legitimate traffic without any malicious activity. **DoS** attacks, such as back, land, neptune, pod, and smurf, aim to overwhelm the target's resources, causing the system to slow down or crash. **U2R** attacks, including buffer_overflow, loadmodule, perl, and rootkit, occur when an attacker gains access to a regular user account and then exploits vulnerabilities to obtain root-level privileges. **R2L** attacks, such as ftp_write, guess_passwd, imap, multihop, phf, spy, warezclient, and warezmaster, occur when an attacker tries to gain unauthorized access to a target system from a remote location. Lastly, **Probe** attacks, such as ipsweep, nmap, portsweep, and satan, involve scanning networks to gather information about vulnerabilities that can be exploited later. These categories represent the key types of attacks present in the NSL-KDD dataset.

Table 4
NSL-KDD multiclass classification.

Cases	Validation accuracy	Test accuracy
All features	99.73%	87.97%
XGBoost features	99.10%	86.38%
First 22 features	96.29%	84.09%

Table 5
Comparison of ML methods on binary and multiclass classification based on NSL-KDD.

ML method	Classification	Test accuracy
MCA-LSTM [45]	Binary	80.52%
CNN-LSTM [46]	Binary	74.77%
Simple RNN [3]	Binary	83.70%
LSTM [3]	Binary	88.13%
GRU [3]	Binary	84.66%
ICNN-ID	Binary	90.75%
MCA-LSTM [45]	Multiclass	82.15%
CNN-LSTM [46]	Multiclass	68.78%
GRU [47]	Multiclass	82.87%
Semi-LSTM	Multiclass	82.21%
LSTM [47]	Multiclass	82.78%
LSTM [48]	Multiclass	82.00%
DNN [49]	Multiclass	75.75%
Simple RNN [3]	Multiclass	84.03%
LSTM [3]	Multiclass	85.93%
GRU [3]	Multiclass	85.65%
ICNN-ID	Multiclass	87.97%

The experimental results are shown in Table 4, which further confirm that feature selection cannot guarantee a 100% improvement in performance. The confusion matrix of the proposed IDS using all features is presented in Fig. 7. It can be observed that data imbalance affects the detection of intrusions in the probe and u2r categories, which is a common issue in other methods and requires further investigation in the future.

Here, we also compare the test accuracy of ICNN-ID with other existing algorithms, and the comparisons are shown in Table 5. The compared methods include long short-term memory network (MCA-LSTM) [45], long-short-term memory based convolutional neural networks [46], Long-Short Term Memory (LSTM) [3,47,48], Gated Recurrent Unit (GRU) [3,47], Simple RNN [3], Semantic parsing based LSTM model [50], and Deep Neural Network [49]. Table 5 demonstrates that ICNN-ID outperforms the compared methods in both binary and multi-class classification tasks.

4.2.2. CICIoV2024

What sets the CICIoV2024 dataset apart is its comprehensive nature: it not only provides detailed network traffic data but also includes metadata about the conditions under which the data was collected. This allows researchers to understand the context of each scenario fully, facilitating more accurate modeling and analysis of IoV security solutions. The dataset is thus instrumental for researchers focusing on pattern recognition, anomaly detection, and the development of AI-driven security measures tailored to the dynamic and intricate nature of modern IoV systems [43]. In this study, we consider the CICIoV2024 decimal dataset, which consists of nine features. This dataset contains 1.2 million records of normal (benign) network communication and 180,000 attack records. The data needs to be balanced. To achieve this, we shuffle the normal records and select 200,000 of them for training and testing. After combining and shuffling the normal and attack records into a single dataset, we split it into training and test sets with an 80:20 ratio, as used in [43]. We do not use a validation set for tuning the hyperparameters of the model. A similar LeNet architecture is used, with some minor modifications to adjust the output to six classes, as required by the CICIoV2024 dataset.

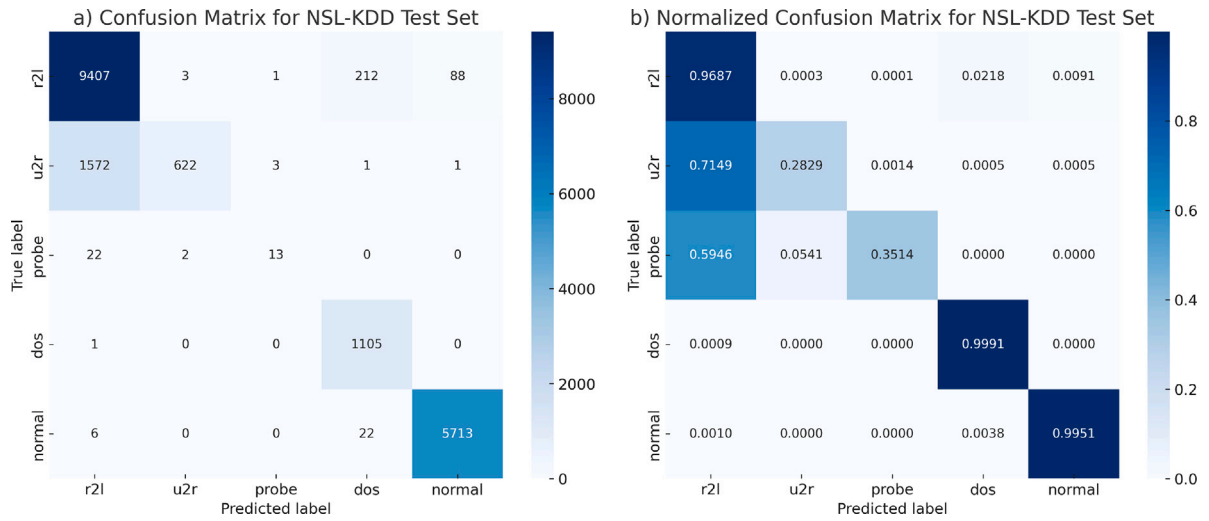


Fig. 7. Confusion matrices for NSL-KDD test set using ICNN-ID based on all features.

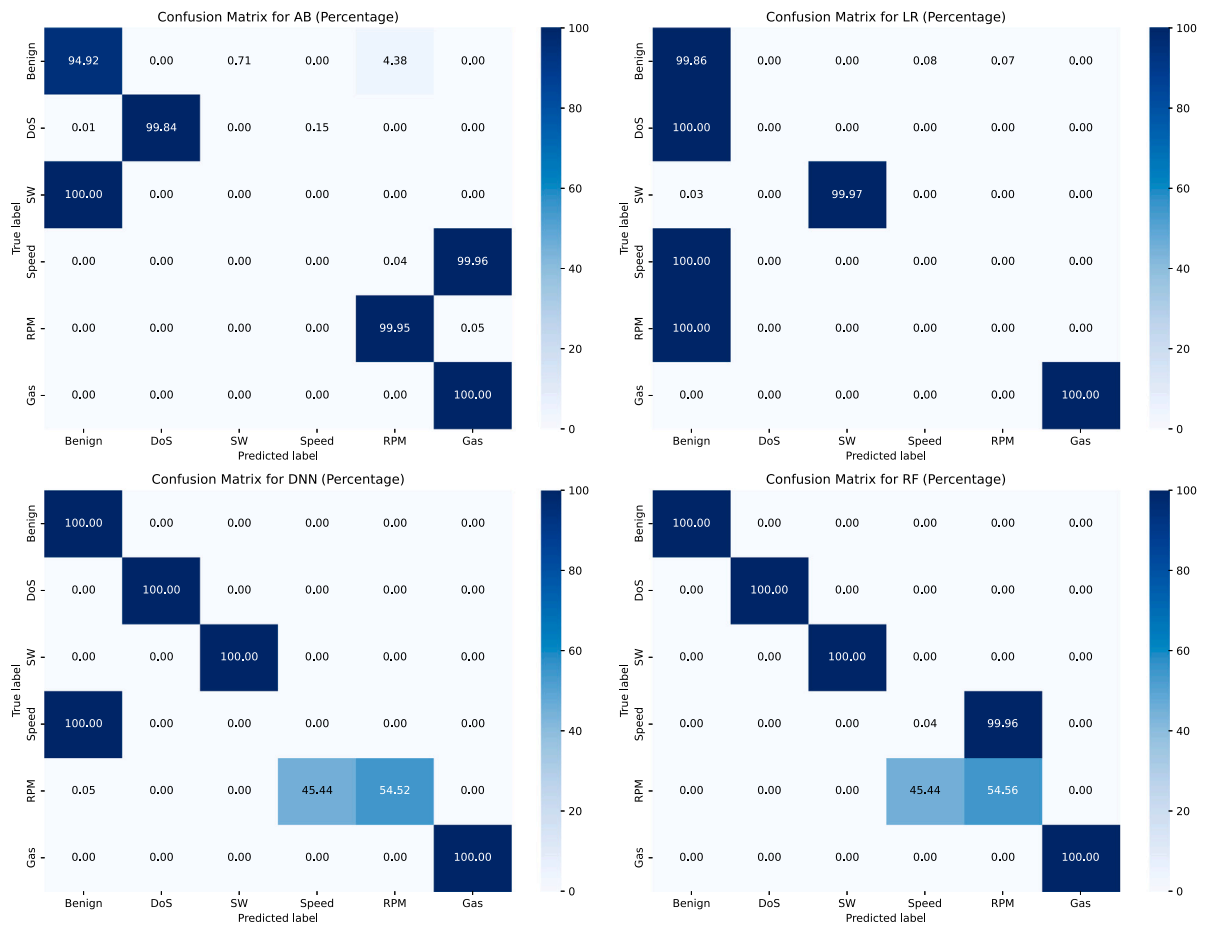


Fig. 8. Confusion matrix for CICIoV2024 using the existing methods (AB, LR, DNN and RF).

In Ref. [43], four classical machine learning algorithms, which are Adaboost (AB), Logistic Regression (LR), Deep Neural Network (DNN), and Random Forest (RF), were applied to CICIoV2024, and their normalized/percentage confusion matrixes are shown in Fig. 8. For ICNN-ID, its confusion matrix is given in Fig. 9. The setting of AB, LR, DNN and RF can be seen in [43]. The test accuracy of AB, LR, DNN, RF, and ICNN-ID is presented in Table 6.

4.3. Integration of EfficientNet-B0 for NSL-KDD multiclass classification

Recent advancements in CNNs have led to the development of more sophisticated architectures such as ResNet and EfficientNet [51], which offer good feature extraction capabilities and computational efficiency. Traditional CNNs like LeNet, while effective for fundamental image classification tasks, may not fully exploit complex patterns in modern



Fig. 9. Confusion matrix for CICIoV2024 using ICNN-ID.

Table 6
Comparison of ML methods on multiclass classification based on CICIoV2024.

ML method	TAC
Adaboost [43]	92.38%
Logistic Regression [43]	88.90%
Deep Neural Network [43]	96.46%
Random Forest [43]	96.46%
ICNN-ID	99.996%

intrusion detection datasets. To explore the potential benefits of using an advanced CNN for intrusion detection, we employ EfficientNet-B0, a state-of-the-art architecture that introduces compound scaling to balance network depth, width, and resolution efficiently. EfficientNet-B0 is designed to achieve high accuracy with fewer parameters, making it an attractive alternative for network intrusion detection tasks. This section details how we adapted EfficientNet-B0 for the NSL-KDD dataset and compares its performance to LeNet in a multiclass classification setting. Accuracy and the confusion matrix are used to evaluate its performance in this section.

Since EfficientNet-B0 was originally designed for three-channel RGB images, several modifications were required to adapt it for NSL-KDD intrusion detection, where input data consists of single-channel structured features. To accommodate this, we modified the first convolutional layer to accept single-channel grayscale inputs while maintaining the model's ability to extract complex spatial features. Additionally, as EfficientNet-B0 is optimized for 32×32 pixel images, we rescaled the NSL-KDD feature representations from 28×28 to 32×32 using an upsampling technique to ensure compatibility. The final fully connected layer was adjusted to five output classes ($num_classes = 5$), corresponding to the NSL-KDD multiclass classification task. Given that EfficientNet architectures can be sensitive to high learning rates, we fine-tuned the model by setting the initial learning rate to 0.001 and employing the Adam optimizer to optimize convergence stability. These modifications ensure that EfficientNet-B0 is effectively adapted to the domain of network intrusion detection while maintaining computational efficiency.

To assess the effectiveness of EfficientNet-B0 for intrusion detection, we compared its performance to the LeNet architecture on the NSL-KDD dataset. Our experimental results show that EfficientNet-B0 achieves a higher classification accuracy (mean TAC = 88.29%) compared to LeNet (mean TAC = 87.97%), demonstrating that more sophisticated CNN architectures can enhance feature extraction in network intrusion detection tasks. While the improvement is marginal in this specific scenario, the potential advantages of EfficientNet-B0 lie in its ability to scale efficiently with more complex datasets and larger-scale applications. This result suggests that when computational cost is not a limiting factor, advanced CNNs such as EfficientNet-B0 can be a more effective alternative to traditional models like LeNet for IDSs.

4.4. Statistical and performance comparison of ICNN-ID

To ensure a comprehensive evaluation of the proposed ICNN-ID model, this section presents a detailed comparative analysis against both traditional and state-of-the-art deep learning baselines. While previous evaluations focused primarily on basic machine learning methods, it is crucial to benchmark ICNN-ID against more recent and competitive models to validate its effectiveness in intrusion detection tasks. All the simulations are based on NSL-KDD due to its characteristics mentioned in Section 2.

In this section, we compare ICNN-ID with 1D CNN, which is a widely used deep learning baseline for IDSs, and DCNNBiLSTM [10], which is a recently proposed deep learning model that integrates deep convolutional neural networks and bidirectional long short-term memory (BiLSTM) for enhanced feature extraction and sequential pattern learning. The 1D CNN IDS model follows a straightforward architecture, consisting of two convolutional layers followed by fully connected layers, allowing it to capture essential patterns in the network traffic data. The input to the model consists of 41 numerical features representing different aspects of network connections. The first layer of the model is a 1D convolutional layer that applies 128 filters with a kernel size of 3. The second convolutional layer builds on the first one, increasing the number of filters to 256. Following the convolutional layers,

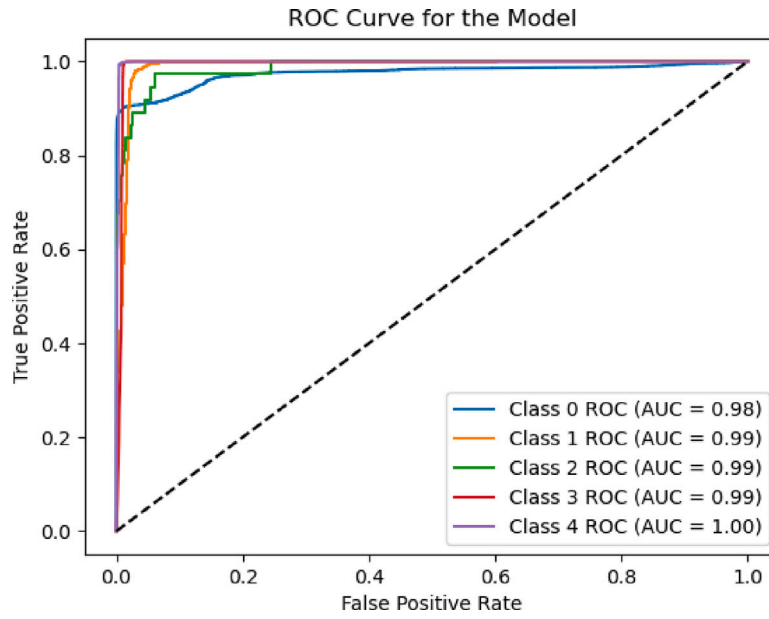


Fig. 10. ROC of ICNN-ID on NSL-KDD.

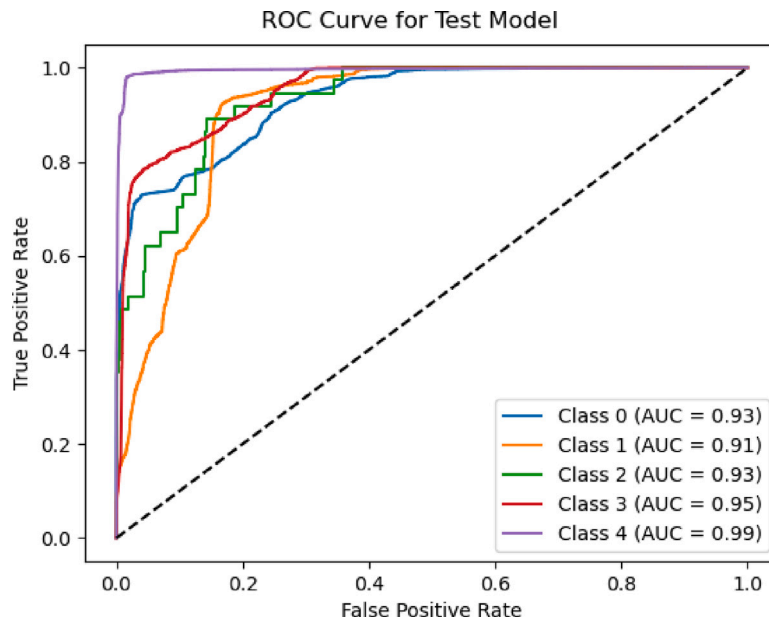


Fig. 11. ROC of 1D CNN on NSL-KDD.

the output is flattened into a one-dimensional vector, and the first fully connected layer consists of 128 neurons, followed by a dropout layer with a probability of 0.1 to prevent overfitting. The second and third fully connected layers contain 64 and 32 neurons, respectively, both followed by dropout layers. The final output layer consists of 5 neurons, corresponding to the five class of NSL-KDD. The model is trained using the Adam optimizer and the initial learning rate is set to 0.0001. The loss function used is CrossEntropyLoss. The training process runs for 30 epochs, but it incorporates an early stopping mechanism to prevent unnecessary computations. Unlike 1D CNN, which directly flattened the CNN output and fed it into fully connected layers, DCNNBiLSTM feeds the convolutional output into two stacked BiLSTM layers. The first BiLSTM Layer has 128 hidden units per direction (256 total) and the second BiLSTM Layer has 256 hidden units per sequence (128 per direction).

The evaluation is conducted using multiple performance metrics, including accuracy, F1-score, Precision, Recall, and ROC curves, to provide a holistic assessment of each model's classification capability. Table 7 give the values of these metrics, while Figs. 10–12 display the ROC plots for these three models. From Table 7, ICNN-ID outperforms both 1D CNN and DCNN-BiLSTM [10] across most metrics. In particular, it achieves the highest accuracy (0.8797) and F1 score (0.8302), as well as notably stronger precision (0.8309) and recall (0.6410). While 1D CNN places second with moderate accuracy (0.8453) and F1 score (0.8089), DCNN-BiLSTM [10] shows the lowest performance, especially in terms of F1 (0.5020) and recall (0.5411). These results indicate that the proposed ICNN-ID architecture provides a more effective balance of precision and recall and, overall, delivers the best classification performance among the compared models. DCNN-BiLSTM did not perform well, likely due to the imbalance in the dataset.

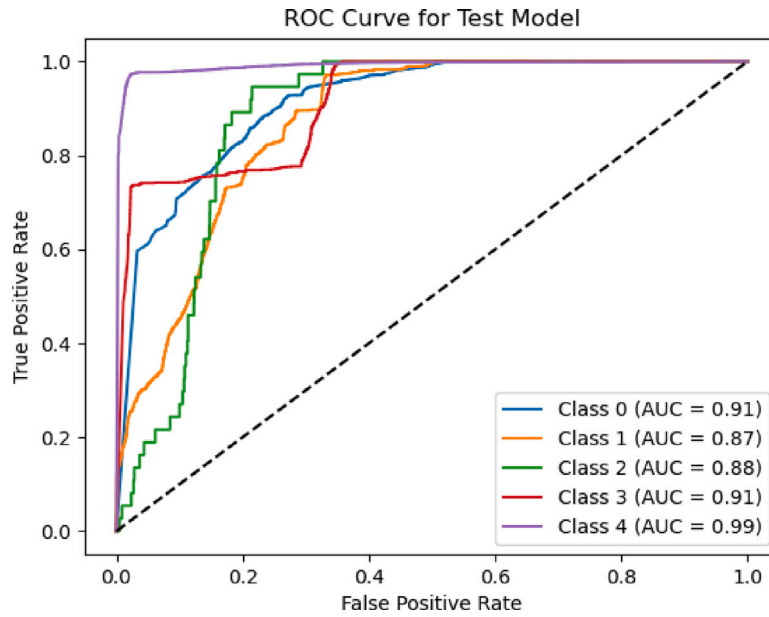


Fig. 12. ROC of DCNN-BiLSTM on NSL-KDD.

Table 7
Performance metrics of different models.

Model	Accuracy	F1 score	Precision	Recall
1D CNN	0.8453 $\pm$ 0.0080	0.6089 $\pm$ 0.0380	0.7807 $\pm$ 0.0276	0.6221 $\pm$ 0.0316
DCNN-BiLSTM [10]	0.8273 $\pm$ 0.0108	0.5030 $\pm$ 0.0229	0.6295 $\pm$ 0.0683	0.5411 $\pm$ 0.0125
ICNN-ID	0.8797 $\pm$ 0.0081	0.6360 $\pm$ 0.0463	0.8309 $\pm$ 0.0561	0.6410 $\pm$ 0.0316

Since the problem involves five different conditions (classes 0 to 4), the function first binarizes the labels by converting each class into a separate binary classification problem when calculating the ROC. In Figs. 10–12, Classes 0, 1, 2, 3, and 4 correspond to *normal*, *r2l*, *u2r*, *probe*, *dos*, respectively. Fig. 10 shows that each class exhibits near-perfect separation using ICNN-ID, with AUC values ranging from 0.98 to 1.00. However, from Fig. 12, while the DCNN-BiLSTM model demonstrates strong classification performance, it has notable drawbacks, particularly in handling certain attack classes with lower AUC scores, such as Class 1 (AUC = 0.87) and Class 2 (AUC = 0.88). Overall, Figs. 10–12 indicate that ICNN-ID exhibits excellent discrimination ability across all attack types as well as normal traffic, with minimal overlap between classes.

Additionally, to ensure statistical validity, we perform t-tests by conducting 15 independent training and testing runs for ICNN-ID, 1D CNN, and DCNN-BiLSTM. The p-values obtained from these tests are analyzed to determine whether the observed performance differences are statistically significant. Paired t-tests on accuracy are conducted to compare ICNN-ID with 1D CNN and DCNN-BiLSTM. We obtained the following p-values: $5.67\text{e-}12$ for ICNN-ID vs. 1D CNN and $5.33\text{e-}14$ for ICNN-ID vs. DCNN-BiLSTM. These p-values further confirm that ICNN-ID outperforms both 1D CNN and DCNN-BiLSTM.

4.5. Model convergence

Analyzing training epochs, convergence behavior, and overfitting detection is crucial in neural network classifiers. Here, based on the multiclass classification of NSL-KDD using ICNN-ID, we obtain two curves (one for validation accuracy and one for test accuracy) plotted against the number of epochs to examine how the model is trained, as shown in Fig. 13. The mean validation accuracy and test accuracy over 40 epochs are 0.99667 and 0.87596, respectively, with standard deviations of 0.00201 and 0.00837.

As seen in Fig. 13, the model exhibits a rapid learning process, achieving the mean validation accuracy of approximately 0.997 and the mean test accuracy 0.876 within the first three to four epochs. This fast convergence indicates that the model efficiently captures the core decision boundaries early in training. Given that the accuracy stabilizes quickly, the overall training cost in terms of epochs is relatively low. This property makes the model highly suitable for online or near-online training scenarios, where frequent adaptation to new data is required.

Overfitting typically manifests as a widening gap between validation and test accuracy, where validation accuracy remains high while test accuracy deteriorates. However, in our case, as shown in Fig. 13, both validation and test accuracy remain stable throughout training. While the validation accuracy is consistently higher than test accuracy, which is an expected phenomenon due to differences in training and test distributions, there is no significant degradation in test accuracy as training progresses. This stability suggests that the model is generalizing well without overfitting, making it robust for practical deployment.

To further optimize training efficiency and prevent unnecessary overfitting, ICNN-ID employs the early stopping technique for the multiclass classification simulations in Section 4.2. The training process was configured with the following settings: Maximum number of epochs: `num_epochs = 40`; Early stopping criteria: Monitor validation loss with `min_delta = 0.001` and `patience = 4`; Stopping condition: Training is terminated early if the validation loss does not decrease by at least 0.001 for four consecutive epochs. This approach prevents excessive training, reducing computational overhead while maintaining optimal performance.

4.6. Discussion

As can be seen from Tables 2–6, ICNN-ID can improve the performance if the AI are properly designed and all the features are fully used, ICNN-ID outperforms the existing algorithms. According to Fig. 7,

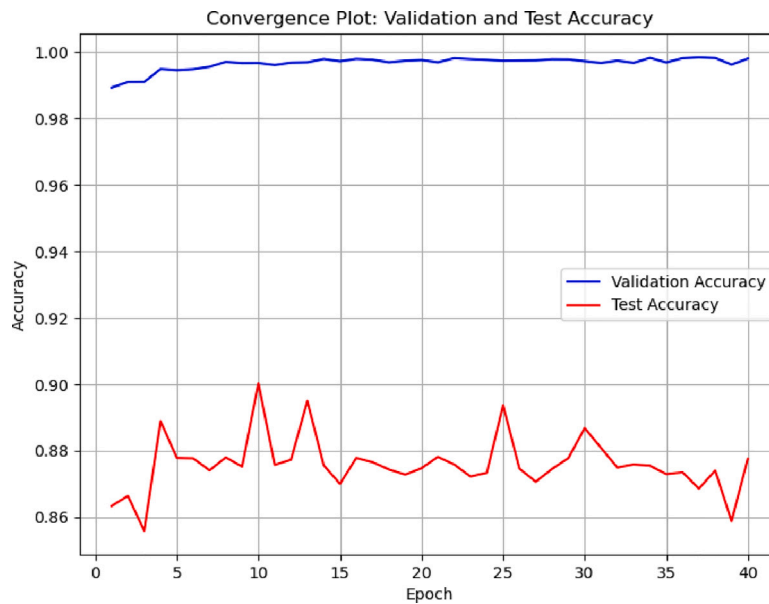


Fig. 13. Model convergence.

the confusion matrix highlights that ICNN-ID performs well in detecting most attack types, but there are notable misclassifications, particularly between u2r and probe attacks. A large number of u2r attacks are incorrectly classified as r2l, and a lot of probe attacks are incorrectly classified as r2l. Regarding the classification performance of u2r and probe attack types, their lower detection rates are primarily attributed to severe class imbalance rather than the absence of time-related features. These attack types are represented by a very small number of samples, making it challenging for the model to learn effective patterns. The imbalance issue is well known in intrusion detection datasets and requires additional techniques such as data augmentation, synthetic sample generation, or cost-sensitive learning, which we identify as areas for future research.

Comparing Figs. 8 and 9, ICNN-ID effectively detects the various attacks, whereas the classical methods fail to detect some of them. The TAC of 99.996% on the CICIoV2024 test set may raise concerns about overfitting or unrealistic performance expectations. However, this high accuracy was obtained using a separate test set, which was completely unseen during training, confirming the generalization ability of ICNN-ID.

Through the comparative analysis presented in Table 7 and Figs. 10–12, we demonstrate that ICNN-ID achieves better classification performance, offering higher accuracy and improved robustness compared to the other models. According to Fig. 13, ICNN-ID exhibits fast and stable convergence, with no obvious signs of overfitting, as the model's validation accuracy remains high, and the test accuracy fluctuates within a narrow range without degradation.

CNN-based approaches, including LeNet, may introduce computational overhead, especially in IoT environments with limited processing power. To address the computational challenges of deploying CNN-based intrusion detection in resource-constrained IoT environments, various optimization techniques can be employed to enhance scalability without significantly compromising accuracy. Model pruning and quantization can reduce the number of redundant parameters and lower numerical precision, respectively, leading to faster inference and reduced memory usage. Additionally, leveraging edge-cloud hybrid architectures allows lightweight feature extraction on edge devices while offloading computationally intensive CNN-based detection to cloud or fog nodes, minimizing latency and bandwidth consumption. Lightweight CNN architectures such as MobileNet, SqueezeNet, and EfficientNet offer efficient alternatives designed specifically for

low-power devices by using depthwise separable convolutions and reducing parameter counts. Furthermore, hardware acceleration techniques, such as deploying models using TensorFlow Lite, TinyML, or OpenVINO on optimized hardware like NVIDIA Jetson Nano or Google Coral, can enhance real-time processing while maintaining energy efficiency. The selection of an appropriate optimization strategy depends on hardware constraints and real-time processing needs, and future work can explore hybrid approaches that combine model compression with adaptive inference techniques to further improve the trade-off between efficiency and detection performance.

5. Conclusion

To fully leverage the power of image classification neural networks, this paper proposes a new intrusion detection framework based on image classification networks. In this approach, a fixed-random data conversion technique was specifically chosen to transform discrete, time-independent network traffic data into a grid structure, facilitating effective processing by CNNs. This approach differs fundamentally from Principal Component Analysis (PCA) and Variational Autoencoders (VAEs), which are designed for dimensionality reduction and feature extraction rather than data format transformation. Our experiments demonstrate that feature extraction-based IDS may not achieve optimal performance if the feature extraction methods are not appropriately selected. However, ICNN-ID shows superior performance compared to some existing methods due to its ability to fully utilize the available features. In future work, we plan to explore the application of this method in other domains using different image classification AI models besides the AI models using convolution operation. Some attacks (e.g., DDoS, port scanning) are inherently time-dependent, and incorporating temporal information can significantly enhance detection performance if available. We encourage researchers and dataset providers to develop and release time-annotated datasets to facilitate the evaluation of intrusion detection models under realistic conditions. For example, each sample or record could include data spanning two or more time slots.

CRediT authorship contribution statement

Yanxia Sun: Writing – original draft, Formal analysis, Conceptualization. **Zenghui Wang:** Writing – review & editing, Writing – original draft, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partly supported by the South African National Research Foundation under Grant nos. CPRR240327211161, CHN231229201835, RA22112976288 and AJCR230704126719.

Data availability

Data will be made available on request.

References

- [1] R. Kaur, D. Gabrijelčič, T. Klobučar, Artificial intelligence for cybersecurity: Literature review and future research directions, *Inf. Fusion* 97 (2023) 101804, <http://dx.doi.org/10.1016/j.inffus.2023.101804>.
- [2] S.M. Kasongo, Y. Sun, A deep learning method with wrapper based feature extraction for wireless intrusion detection system, *Comput. Secur.* 92 (2020) 101752, <http://dx.doi.org/10.1016/j.cose.2020.101752>.
- [3] S.M. Kasongo, A deep learning technique for intrusion detection system using a Recurrent Neural Networks based framework, *Comput. Commun.* 199 (2023) 113–125, <http://dx.doi.org/10.1016/j.comcom.2022.12.010>.
- [4] S. Kasongo, Y. Sun, Performance analysis of intrusion detection systems using a feature selection method on the UNSW-NB15 dataset, *J. Big Data* 7 (2020) 105, <http://dx.doi.org/10.1186/s40537-020-00379-6>.
- [5] M. Behiry, M. Aly, Cyberattack detection in wireless sensor networks using a hybrid feature reduction technique with AI and machine learning methods, *J. Big Data* 11 (2024) 16, <http://dx.doi.org/10.1186/s40537-023-00870-w>.
- [6] Y. LeCun, B. Boser, J.S. Denker, D. Henderson, R.E. Howard, W. Hubbard, L.D. Jackel, Backpropagation applied to handwritten zip code recognition, *Neural Comput.* 1 (4) (1989) 541–551, <http://dx.doi.org/10.1162/neco.1989.1.4.541>.
- [7] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet classification with deep convolutional neural networks, *Commun. ACM* 60 (6) (2017) 84–90, <http://dx.doi.org/10.1145/3065386>.
- [8] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, 2014, arXiv preprint [arXiv:1409.1556](https://arxiv.org/abs/1409.1556).
- [9] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 1–9.
- [10] V. Hnamte, J. Hussain, DCNNBiLSTM: An efficient hybrid deep learning-based intrusion detection system, *Telemat. Inform. Rep.* 10 (2023) 100053, <http://dx.doi.org/10.1016/j.teler.2023.100053>, URL <https://www.sciencedirect.com/science/article/pii/S2772503023000130>.
- [11] A. El-Ghamry, A. Darwish, A.E. Hassanien, An optimized CNN-based intrusion detection system for reducing risks in smart farming, *Internet Things* 22 (2023) 100709, <http://dx.doi.org/10.1016/j.iot.2023.100709>.
- [12] G.W.F. Hegel, *Grundlinien der Philosophie des Rechts*, Suhrkamp, Frankfurt am Main, 1972, p. 11.
- [13] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, SMOTE: Synthetic minority over-sampling technique, *J. Artificial Intelligence Res.* 16 (2002) 321–357.
- [14] H. He, Y. Bai, E.A. Garcia, S. Li, ADASYN: Adaptive synthetic sampling approach for imbalanced learning, in: 2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence), 2008, pp. 1322–1328, <http://dx.doi.org/10.1109/IJCNN.2008.4633969>.
- [15] F. Kamalov, H.-H. Leung, A.K. Cherukuri, Keep it simple: random oversampling for imbalanced data, in: 2023 Advances in Science and Engineering Technology International Conferences, ASET, 2023, pp. 1–4, <http://dx.doi.org/10.1109/ASET56582.2023.10180891>.
- [16] M. Kubat, R.C. Holte, S. Matwin, Machine learning for the detection of oil spills in satellite radar images, *Mach. Learn.* 30 (2–3) (1997) 195–215.
- [17] H. Chen, Y. Liu, J. Zhao, X. Liu, Research on intrusion detection based on BP neural network, in: 2021 IEEE International Conference on Consumer Electronics and Computer Engineering, ICCECE, 2021, pp. 79–82, <http://dx.doi.org/10.1109/ICCECE51280.2021.9342479>.
- [18] H.R. Sayegh, W. Dong, A.M. Al-madani, Enhanced intrusion detection with LSTM-based model, feature selection, and SMOTE for imbalanced data, *Appl. Sci.* 14 (2) (2024) <http://dx.doi.org/10.3390/app14020479>, URL <https://www.mdpi.com/2076-3417/14/2/479>.
- [19] L. De Rose, G. Andresini, A. Appice, D. Malerba, VINCENT: Cyber-threat detection through vision transformers and knowledge distillation, *Comput. Secur.* 144 (2024) 103926, <http://dx.doi.org/10.1016/j.cose.2024.103926>.
- [20] S.S. Qureshi, J. He, S.U. Qureshi, N. Zhu, A. Wajahat, A. Nazir, F. Ullah, A. Wadud, Advanced AI-driven intrusion detection for securing cloud-based industrial IoT, *Egypt. Inform. J.* 30 (2025) 100644, <http://dx.doi.org/10.1016/j.eij.2025.100644>, URL <https://www.sciencedirect.com/science/article/pii/S1110866525000374>.
- [21] U. Ahmed, Z. Jiangbin, S. Khan, M.T. Sadiq, Consensus hybrid ensemble machine learning for intrusion detection with explainable AI, *J. Netw. Comput. Appl.* 235 (2025) 104091, <http://dx.doi.org/10.1016/j.jnca.2024.104091>, URL <https://www.sciencedirect.com/science/article/pii/S1084804524002686>.
- [22] S. A.L., S. Sagiroglu, Explainable artificial intelligence models in intrusion detection systems, *Eng. Appl. Artif. Intell.* 144 (2025) 110145, <http://dx.doi.org/10.1016/j.engappai.2025.110145>, URL <https://www.sciencedirect.com/science/article/pii/S0952197625001459>.
- [23] H. Nandanwar, R. Katarya, Securing Industry 5.0: An explainable deep learning model for intrusion detection in cyber-physical systems, *Comput. Electr. Eng.* 123 (2025) 110161, <http://dx.doi.org/10.1016/j.compeleceng.2025.110161>, URL <https://www.sciencedirect.com/science/article/pii/S0045790625001041>.
- [24] A. Zhang, Y. Zhao, C. Zhou, T. Zhang, ResACAG: A graph neural network based intrusion detection, *Comput. Electr. Eng.* 122 (2025) 109956, <http://dx.doi.org/10.1016/j.compeleceng.2024.109956>, URL <https://www.sciencedirect.com/science/article/pii/S0045790624008814>.
- [25] M. Termos, Z. Ghalmane, M. el Amine Brahmia, A. Fadlallah, A. Jaber, M. Zghal, GDLC: A new Graph Deep Learning framework based on centrality measures for intrusion detection in IoT networks, *Internet Things* 26 (2024) 101214, <http://dx.doi.org/10.1016/j.iot.2024.101214>, URL <https://www.sciencedirect.com/science/article/pii/S2542660524001550>.
- [26] M. Arafah, I. Phillips, A. Adnane, M. Alauthman, N. Aslam, An enhanced BiGAN architecture for network intrusion detection, *Knowl.-Based Syst.* 314 (2025) 113178, <http://dx.doi.org/10.1016/j.knsys.2025.113178>, URL <https://www.sciencedirect.com/science/article/pii/S0950705125002254>.
- [27] M.A. Alkhonaini, M.A. Alohal, M. Aljebreen, M.M. Eltahir, M.H. Alanazi, A. Yafaz, R. Alsini, A.O. Khadidos, Sandpiper optimization with hybrid deep learning model for blockchain-assisted intrusion detection in iot environment, *Alex. Eng. J.* 112 (2025) 49–62, <http://dx.doi.org/10.1016/j.aej.2024.10.032>, URL <https://www.sciencedirect.com/science/article/pii/S1110016824011864>.
- [28] Y. Li, Z. Li, M. Li, A comprehensive survey on intrusion detection algorithms, *Comput. Electr. Eng.* 121 (2025) 109863, <http://dx.doi.org/10.1016/j.compeleceng.2024.109863>, URL <https://www.sciencedirect.com/science/article/pii/S0045790624007900>.
- [29] R. Chinnamasamy, M. Subramanian, S.V. Easwaramoorthy, J. Cho, Deep learning-driven methods for network-based intrusion detection systems: A systematic review, *ICT Express* 11 (1) (2025) 181–215, <http://dx.doi.org/10.1016/j.icte.2025.01.005>, URL <https://www.sciencedirect.com/science/article/pii/S2405959525000050>.
- [30] KDDCup1999, KDD cup 1999 data, 1999, <https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>. (Accessed 06 September 2024).
- [31] V. Hnamte, J. Hussain, Dependable intrusion detection system using deep convolutional neural network: A novel framework and performance evaluation approach, *Telemat. Inform. Rep.* 11 (2023) 100077, <http://dx.doi.org/10.1016/j.teler.2023.100077>.
- [32] Y. Zhu, Y. Wang, L. Zhou, Y. Xia, FC-Trans: Deep learning methods for network intrusion detection in big data environments, *Comput. Secur.* 154 (2025) 104392, <http://dx.doi.org/10.1016/j.cose.2025.104392>, URL <https://www.sciencedirect.com/science/article/pii/S0167404825000811>.
- [33] S.S. Bamber, A.V.R. Katkuri, S. Sharma, M. Angurala, A hybrid CNN-LSTM approach for intelligent cyber intrusion detection system, *Comput. Secur.* 148 (2025) 104146, <http://dx.doi.org/10.1016/j.cose.2024.104146>, URL <https://www.sciencedirect.com/science/article/pii/S0167404824004516>.
- [34] Y. Xue, C. Kang, H. Yu, HAE-HRL: A network intrusion detection system utilizing a novel autoencoder and a hybrid enhanced LSTM-CNN-based residual network, *Comput. Secur.* 151 (2025) 104328, <http://dx.doi.org/10.1016/j.cose.2025.104328>, URL <https://www.sciencedirect.com/science/article/pii/S0167404825000173>.
- [35] M. C. G.N.K.S. Babu, A PCA + RF-VI hybrid approach for feature extraction in intrusion detection systems, in: 2024 Ninth International Conference on Science Technology Engineering and Mathematics, ICONSTEM, 2024, pp. 1–8, <http://dx.doi.org/10.1109/ICONSTEM60960.2024.10568604>.
- [36] S.A. Al-shalif, N. Senan, F. Saeed, W. Ghaban, N. Ibrahim, M. Aamir, W. Sharif, A systematic literature review on meta-heuristic based feature selection techniques for text classification, *PeerJ Comput. Sci.* 10 (2024) e2084, <http://dx.doi.org/10.7717/peerj-cs.2084>, URL DOI: 10.7717/peerj-cs.2084.
- [37] X. Yang, H. Che, M.-F. Leung, Tensor-based unsupervised feature selection for error-robust handling of unbalanced incomplete multi-view data, *Inf. Fusion* 114 (2025) 102693, <http://dx.doi.org/10.1016/j.inffus.2024.102693>, URL <https://www.sciencedirect.com/science/article/pii/S1566253524004718>.

- [38] S. Zhou, P. Song, Y. Yu, W. Zheng, Structural regularization based discriminative multi-view unsupervised feature selection, *Knowl.-Based Syst.* 272 (2023) 110601, <http://dx.doi.org/10.1016/j.knosys.2023.110601>, URL <https://www.sciencedirect.com/science/article/pii/S0950705123003519>.
- [39] J. Zhang, Y. Qiu, L. Peng, Q. Zhou, Z. Wang, M. Qi, A comprehensive review of methods based on deep learning for diabetes-related foot ulcers, *Front. Endocrinol.* 13 (2022) 945020, <http://dx.doi.org/10.3389/fendo.2022.945020>.
- [40] M.T. Islam, I.H. Lee, A.I. Alzahrani, K. Muhammad, MEXFIC: A meta ensemble explainable approach for AI-synthesized fake image classification, *Alex. Eng. J.* 116 (2025) 351–363, <http://dx.doi.org/10.1016/j.aej.2024.12.031>, URL <https://www.sciencedirect.com/science/article/pii/S111001682401617X>.
- [41] A.S. Remigio, IncARMAG: A convolutional neural network with multi-level autoregressive moving average graph convolutional processing framework for medical image classification, *Neurocomputing* 617 (2025) 129038, <http://dx.doi.org/10.1016/j.neucom.2024.129038>, URL <https://www.sciencedirect.com/science/article/pii/S0925231224018095>.
- [42] J. Tmamna, E. Ben Ayed, R. Fourati, A. Hussain, M. Ben Ayed, A CNN pruning approach using constrained binary particle swarm optimization with a reduced search space for image classification, *Appl. Soft Comput.* 164 (2024) 111978, <http://dx.doi.org/10.1016/j.asoc.2024.111978>, URL <https://www.sciencedirect.com/science/article/pii/S156849462400752X>.
- [43] E.C.P. Neto, H. Taslimasa, S. Dadkhah, S. Iqbal, P. Xiong, T. Rahman, A.A. Ghorbani, CICIoV2024: Advancing realistic IDS approaches against DoS and spoofing attack in IoV CAN bus, *Internet Things* 26 (2024) 101209, <http://dx.doi.org/10.1016/j.iot.2024.101209>.
- [44] Y. Lecun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, *Proc. IEEE* 86 (11) (1998) 2278–2324, <http://dx.doi.org/10.1109/5.726791>.
- [45] R. Dong, X. Li, Q. Zhang, H. Yuan, Network intrusion detection model based on multivariate correlation analysis-long short-time memory network, *IET Inf. Secur.* 14 (2) (2019) 166–174.
- [46] C.-M. Hsu, H.-Y. Hsieh, S. Prakosa, M.Z. Azhari, J.-S. Leu, Using long-short-term memory based convolutional neural networks for network intrusion detection, in: *Wireless Internet*, Springer Cham, 2019, pp. 86–94, http://dx.doi.org/10.1007/978-3-030-06158-6_9, International Conference on Wireless Internet (WICON), 2018.
- [47] Z. Li, A. Rios, G. Xu, L. Trajković, Machine learning techniques for classifying network anomalies and intrusions, in: *IEEE International Symposium on Circuits and Systems, ISCAS, IEEE*, 2019, pp. 1–5.
- [48] S. Wang, C. Xia, T. Wang, A novel intrusion detector based on deep learning hybrid methods, in: *5th International Conference on Big Data Security on Cloud, IEEE International Conference on High Performance and Smart Computing, HPSC, IEEE*, 2019, pp. 300–305.
- [49] T. Tang, L. Mhamdi, D. McLernon, S. Zaidi, M. Ghogho, Deep learning approach for network intrusion detection in software defined networking, in: *International Conference on Wireless Networks and Mobile Communications, WINCOM, IEEE*, 2016, pp. 258–263.
- [50] Z. Li, Z. Qin, A semantic parsing based LSTM model for intrusion detection, in: *International Conference on Neural Information Processing, Springer, Cham*, 2018, pp. 600–609.
- [51] K. Kansal, T.B. Chandra, A. Singh, ResNet-50 vs. EfficientNet-B0: Multi-centric classification of various lung abnormalities using deep learning, *Procedia Comput. Sci.* 235 (2024) 70–80, <http://dx.doi.org/10.1016/j.procs.2024.04.007>, International Conference on Machine Learning and Data Engineering (ICMLDE 2023), URL <https://www.sciencedirect.com/science/article/pii/S1877050924006768>.